Conference Talk RAG

Live site: https://jrj221.github.io/conference-rag/

Search and ask questions about General Conference talks (2015–2019). Three ways to search: **keyword**, **semantic** (embeddings), and **Ask (RAG)**, which answers with GPT-4o and cites the talks it used. Logged-in users also get a **History** tab of their past RAG questions. Anyone can open the site and browse the tabs (each tab view is logged to a public `page_views` table), but searching and history are rejected by the database unless you are signed in.

Stack: React + TypeScript (Vite) · Supabase (Postgres + pgvector, Auth, Edge Functions) · OpenAI (`text-embedding-3-small`, `gpt-4o`)

Deliverables

GitHub repository

https://github.com/jrj221/conference-rag

Live deployment

https://jrj221.github.io/conference-rag/

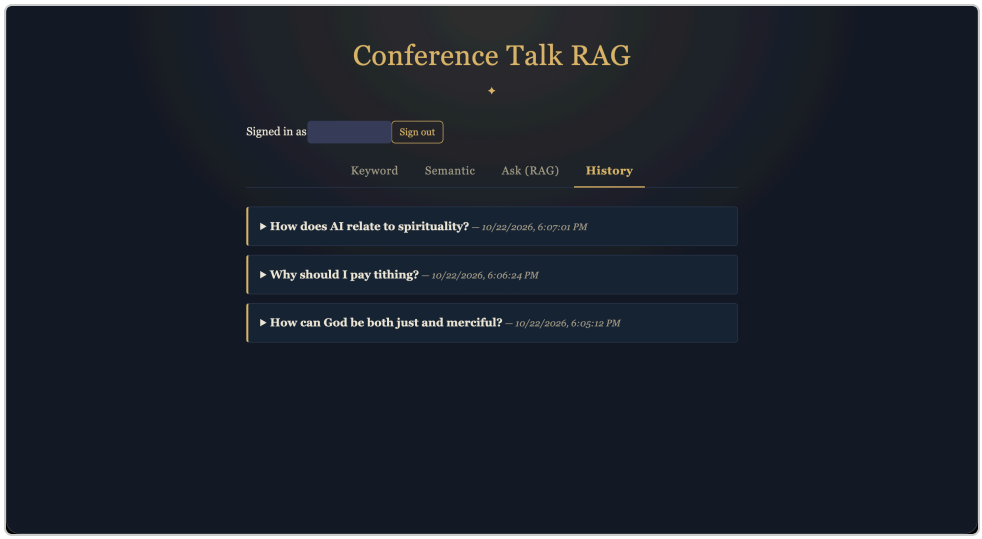
Working RAG query and answer

I asked "How can I forgive?" in the **Ask (RAG)** tab. The answer is built only from the retrieved talks, and it names the talk and speaker it used, here Elder Kevin R. Duncan's "The Healing Ointment of Forgiveness."



Custom feature: question history

I added a **History** tab that saves every question I ask in the Ask (RAG) tab, along with its answer and the talks it was based on. I wanted users to be able to come back to questions they had asked in the past and keep the talk references, instead of having to ask again if they have the same question or want to reconsult the information. It also saves time and OpenAI cost, since reopening a saved answer doesn't call the model again.



How it works:

- Each answer is saved to a new Supabase table, `question_history` (question, answer, the list of source talks, and a timestamp).
- The table is private per user with Row Level Security: policies compare each row's `user_id` with the signed-in user, so people can only read, add, or delete their own history.
- Questions persist across sessions and devices because they live in the database, not in the browser. Each entry expands to show the answer and its sources, and can be deleted.

---

1. Security Architecture

```
Browser (React app)                          Supabase
holds only:                          +-----------------------------------------+
  - anon key           -------->     | Auth: magic link -> user JWT            |
  - user's JWT         -------->     | Postgres: grants + Row Level Security   |
                       -------->     | Edge Function: verifies JWT --> OpenAI  |
                                     |      ^                                  |
                                     |      +-- OPENAI_API_KEY (secret / env)  |
                                     +-----------------------------------------+

My laptop only (git-ignored config.secret.json):
  service/secret key, access token, OpenAI key
```

Asset	What protects it
Database rows	Grants and Row Level Security. Even a valid anon key only gets what a policy allows.
Per-user data (`question_history`)	RLS policies compare each row's `user_id` with the logged-in user (`auth.uid()`).
OpenAI API key	Stored as a Supabase secret and read by Edge Functions as an environment variable. It never reaches the browser.
Edge Functions	Verify the caller's JWT before spending money on OpenAI.
Service/secret key and access token	Kept only in `config.secret.json` on my machine, which is git-ignored.

Why is the anon key safe to expose? The anon key identifies my app as a Supabase project, and requests made with it only get the `anon` Postgres role. It is not a secret, because it grants nothing by itself: RLS and table grants decide what the `anon` role can do. In this project it can only read `page_views`. When a user signs in with a magic link, their requests get an `authenticated` role (and a JWT with their user id), which unlocks the talk data and their own history.

What if someone got the service role key? The service role key acts as an admin and **bypasses RLS entirely**, which makes every policy I wrote useless. Anyone holding it could read, modify, or delete every row in every table, read all users' emails and question history, and generate logins for any user. That is why it only lives in `config.secret.json` for my import scripts and never goes into the frontend or git.

How do Edge Functions protect the OpenAI key? The key is stored in Supabase as a secret (`supabase secrets set`) and the function reads it at runtime as an environment variable (`Deno.env.get`). The key is not in my code, my repo, or the JavaScript sent to the browser. The browser only sends a question and receives an embedding or an answer.

What role does RLS play? RLS is the database-level rule layer: policies written in SQL decide which rows each role or user can read and write. Because the database enforces them, they hold even if my application code has a bug. It is also what lets the frontend talk to the database directly with a public key.

---

2. Edge Functions & the "Secure Middle"

Anything in the browser can be read by the user: the JavaScript, the network requests, the keys. If the browser called OpenAI directly, my OpenAI key would be visible to anyone with DevTools, and an attacker could use it to spend all my money. Supabase's RLS protects my *database*, but it cannot protect a *third-party* key like OpenAI's, so I had to build a secure layer myself.

```
Browser --(question + user's JWT)--> Edge Function --(OPENAI_API_KEY)--> OpenAI
   ^                                    |  1. verify the JWT with Supabase Auth
   +--------(embedding / answer)--------+  2. validate the input
                                           3. call OpenAI
```

When I'm signed in, the browser sends my JWT in the `Authorization` header. The function checks it with Supabase Auth before doing anything. Without that check, anyone who found the function's URL could call

it and run up my OpenAI bill. I tested this: requests with no login, or with only the public anon key, get a 401.

How this applies to production apps. Any app that needs a secret third-party key (OpenAI, Stripe, email or SMS providers) puts a small server-side layer between the browser and that API, which is the same pattern. This architecture also scales without a rewrite. A production app can move to a higher paid Supabase tier to support more users, storage, and compute, while the security model stays the same: RLS for database authorization, JWT-verified Edge Functions, and secrets kept server-side. The upgrade changes capacity, not how access is protected.

---

3. From SQL to Semantics

- **Keyword search (SQL `ILIKE`)** is better when I know the exact words I want. It finds a specific talk by a phrase I remember, like "Think Celestial". Every word I type has to appear in the sentence.
- **Semantic search (vector cosine similarity)** is better when I care about an *idea* and not the exact wording.

Example where each wins. Searching "the eye of faith" with keyword search finds sentences that use that biblical phrase. Searching it semantically returns sentences about seeing and having faith, which may not be what I wanted. In the other direction, a query like "How can I find peace when I am grieving?" fails with keyword search (the sentences that help may never say "peace" or "grieving"), but semantic search returns sentences like "Turn to Him for answers and for comfort" and "You will find healing for that which needs healing."

What makes semantic search "understand" meaning? An embedding model turns each sentence into a list of 1,536 numbers, trained so that sentences with similar meanings end up close together as vectors. My question is turned into a vector by the same model, and the database (pgvector) finds the stored sentence vectors closest to it using cosine similarity. It compares the meaning of whole sentences, not just matching words.

---

4. RAG vs Fine-Tuning

Fine-tuning means continuing to train an existing model on my own examples, which changes the model's weights. It is best when I want to shape *behavior*, like a specific tone or a fixed output format. **RAG** leaves the model unchanged and instead looks up relevant text at question time and puts it in the prompt. It is best for supplying *knowledge*, like specific quotes.

	RAG	Fine-tuning
Best for	Facts, documents, specific quotes	Tone, output format, specialized behavior
Adding new data	Add rows to the database	Retrain the model
Sources / citations	Yes, I know which talks were used	No, knowledge is blended into the weights
Hallucination	Reduced, since answers are grounded in retrieved text	Not reduced, a fine-tuned model can still make things up
Cost	Cheap setup (embedding my data cost about 3 cents)	Training cost, and retraining for every update

Why RAG here: I wanted answers that quote real talks and say which talk each came from, which fine-tuned models are bad at. RAG is also easier and cheaper, and when a new conference happens I only need to scrape and embed the new talks.

When fine-tuning would be better: when I want the model to behave a certain way, such as always answering in a particular tone or always returning a strict format like JSON, or a general chatbot for church questions with a consistent voice, where tracing sources isn't the point. The two can also be combined: fine-tune for behavior and use RAG for the facts.

---

5. AI-Assisted Development

The AI helped me break the assignment into steps, explained the concepts behind each one (JWTs, RLS, embeddings, edge functions), and wrote and ran most of the scripts and the frontend. It is good at explaining things and rephrasing when I had follow-up questions.

I did have to guide it in several places:

- **Search modes.** It first added an "exact phrase" tab next to keyword search. I pushed back that those two are basically the same, and we changed the tabs to Keyword, Semantic, and Ask (RAG).
- **The `page_views` table.** It seeded the table with fake rows, and I questioned why, because I thought the site should log real visits. It turned out the seed was a guess on its part and not in the assignment.
- **Storage limits.** When Supabase emailed me that I was over the free storage limit, I decided to delete half the data, and we kept only the 2015–2019 conferences. The project also had to be rebuilt on a new Supabase project after the database went down.
- **Checking its claims.** I asked whether it had set up the magic-link redirect (it hadn't, and it couldn't have), and I asked follow-up questions when something didn't add up, such as whether a stolen JWT could be reused.

What I learned: AI is very good at explaining concepts, but it makes assumptions that I need to catch, so I should question its choices and check what it actually did. One area of growth is getting it to give shorter, more concise answers.